

[스마트폰으로 제어하는 FDD 음악]

[Smartphone Controlled Floppy Disk Drive Music]

권택준

2019 년 06 월 10 일

요 약

본 문서는 USB 플래시메모리, 클라우드 서비스의 등장으로 저장매체로서의 입지를 완전히 잃어버려서 현재는 수요가 없는 **플로피 디스크 드라이브(Floppy Disk Drive)**를 재탄생시켜, FDD의 단점 중 하나였던 노이즈를 역이용하여 아름다운 소리를 출력하게 하는 기술에 대한 원리를 설명하며, 이 기술을 토대로 스마트폰 블루투스 통신으로 제어하는 새로운 작품을 설계하고 제작합니다.

목 차

I.	서 론	4
1.1	배경	4
II.	사용 이론 및 기술	4
2.1	스마트폰을 통한 연주 데이터 전송	4
2.2	블루투스 통신	8
2.3	FDD로 소리 내기	9
2.4	타이머 인터럽트	11
III.	작품 구현 및 실험결과	13
3.1	작품 설계	13
3.2	작품 제작	14
IV.	결론	15
V.	참고문헌	16

그림 목 차

<그림 1.1>		4
<그림 2.1>		5
<그림 2.2>		6
<그림 2.3>		6
<그림 2.4>		7
<그림 2.5>		7
<그림 2.6>		8
<그림 2.7>		9
<그림 2.8>		9
<그림 2.9>		10
<그림 2.10>		11
<그림 3.1>		13
<그림 3.2>		14
<그림 4.1>		15

I. 서론

1. 배경

FDD는 1971년 IBM에 의해 개발된 저장매체인 플로피 디스크 드라이브(Floppy Disk Drive)의 줄임말입니다. 80~90년대에 본격적으로 보편화되어서 그 당시에는 가장 많이 쓰이던 이동식 보조기억장치였으나 디지털 매체의 급속한 발달로 인해 점차 도태되어 이제는 역사의 뒤편길로 사라져버리고 말았습니다.



<그림 1.1>

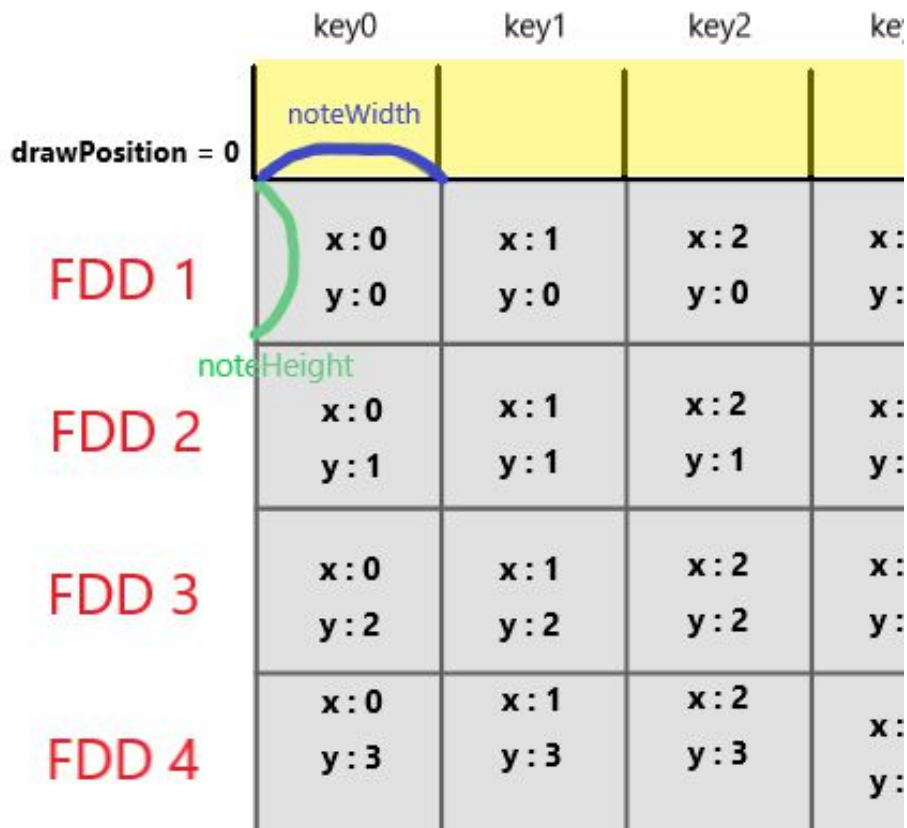
분명 그렇게 잊혀져가고 있었을 터인데, 어느 날 흥미로운 영상을 보게되었습니다. 그것은 여러 개의 플로피 디스크 드라이브가 기괴한 노이즈 소리를 내며 스타워즈 테마곡을 연주하는 영상이었습니다. 이걸 모순된 표현이지만 그 아름다운 잡음은 저를 향수에 젖게 만들기에 충분했습니다. 초등학교 컴퓨터 교육시간 이후로는 다시 볼 일이 없을 줄 알았었는데, 단점이었던 소음을 장점으로 바꿔서 다시 돌아와 노래하는 모습에 깊은 감명을 받아 스마트폰으로 제어해서 연주하는 플로피 디스크 드라이브를 제작해보기로 결심했습니다.

II. 사용 이론 및 기술

1. 스마트폰을 통한 연주 데이터 전송

연주 데이터를 저장하기 위해 Android에서 전용 문법으로 제공하는 SparseArray 를 사용했습니다. SparseArray는 키(key)와 값(value)를 묶어서 하나의 데이터 (entry)로 저장하는 해시맵(HashMap)과 비슷하지만, HashMap은 내부 해시 값에 따라 키순서가 정해지기 때문에 순서를 보장하지 않는 반면에, SparseArray는 키값을 오름차순으로 정렬해서 저장할 수 있고 배열같이 인덱스로 해당 entry를 받아올 수도 있기 때문에 시간에

따라 드문드문 있는 연주 데이터를 저장하고 순서대로 받아와서 송신시키기에는 최적의 데이터 구조라고 생각했습니다.



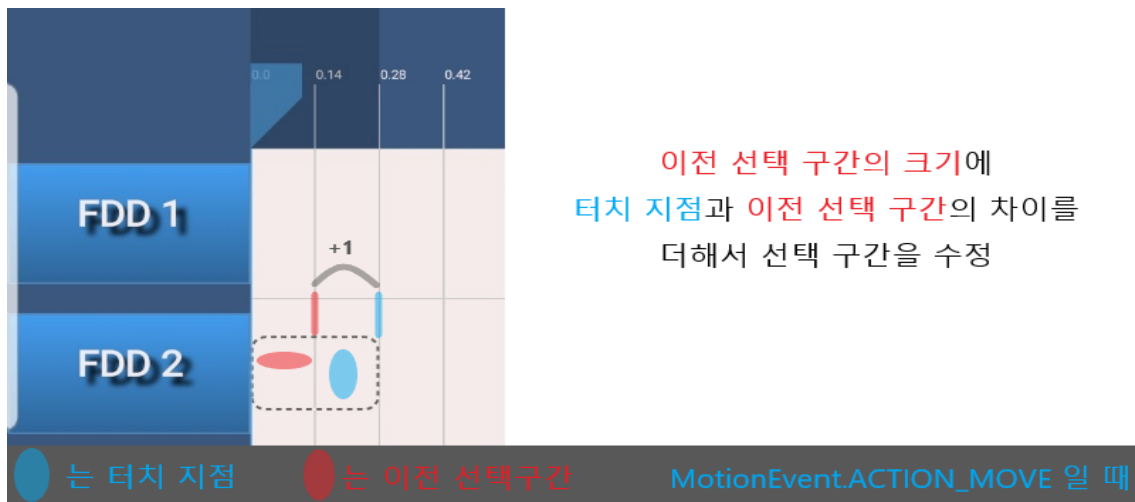
<그림 2.1>

각각의 FDD는 SparseArray를 하나씩 사용하며 노란 영역을 NoteLineImageView, 회색 영역을 NoteTableImageView, 가상의 셀 하나의 높이를 noteHeight, 너비를 noteWidth, 현재 그려야할 시작점의 위치를 drawPosition이라고 정했을 때, NoteLineImageView에 터치 이벤트를 등록해서 NoteLineImageView를 드래그하면 drawPosition의 값이 변하게 하고, 이 값에 따라 NoteTableImageView, NoteLineImageView를 갱신시켜줌으로써 횡스크롤뷰와 같은 움직임을 구현합니다. 횡스크롤뷰를 쓰지 않고 굳이 이런 작업을 해주는 이유는 이미지뷰를 필요한 만큼만 그려주어서 자원의 사용을 최소화시키기 위함입니다. NoteTableImageView에도 터치 이벤트를 등록 해서 터치한 셀의 위치(x,y)를 찾아내게 하는데, 여기서 x의 값은 터치한 NoteTableImageView의 X축 좌표 값과 drawPosition을 더한 값을 noteWidth로 나눈 값, y의 값은 NoteTableImageView의 Y축 좌표 값을 noteHeight로 나눈 값입니다.



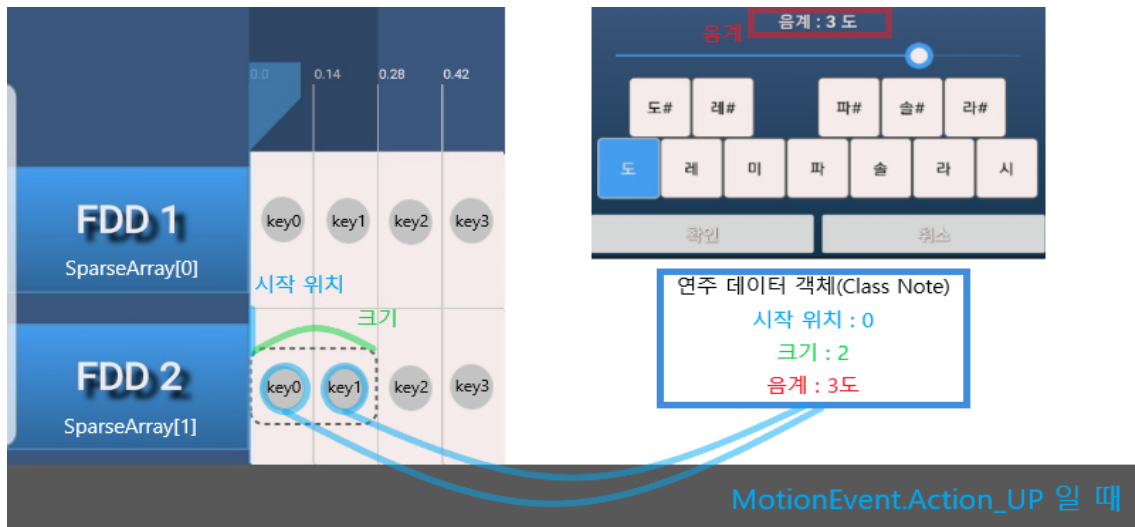
<그림 2.2>

사용자는 터치한 셀의 위치(x,y)를 토대로 선택 구간(x,y,크기)의 값을 지정해야 하는데, 여기서 x와 y의 값은 처음 터치되었을 때(MotionEvent.ACTION_DOWN) 이미 정해졌으므로 그대로 넣고, 크기는 1로 초기화 합니다.



<그림 2.3>

사용자는 터치 지점을 움직여가면서(MotionEvent.ACTION_MOVE) 선택 구간의 크기를 결정하게 되고,



<그림 2.4>

손가락을 떼면(Motion Event.ACTION_UP) 선택 구간에 연주 데이터를 설정하는 창을 띄우고, 설정을 마치고 확인을 누르면 연주 데이터 객체를 생성(new Note)한 후, 해당 구간의 SparseArray에 (key, 연주데이터) 쌍을 추가합니다.

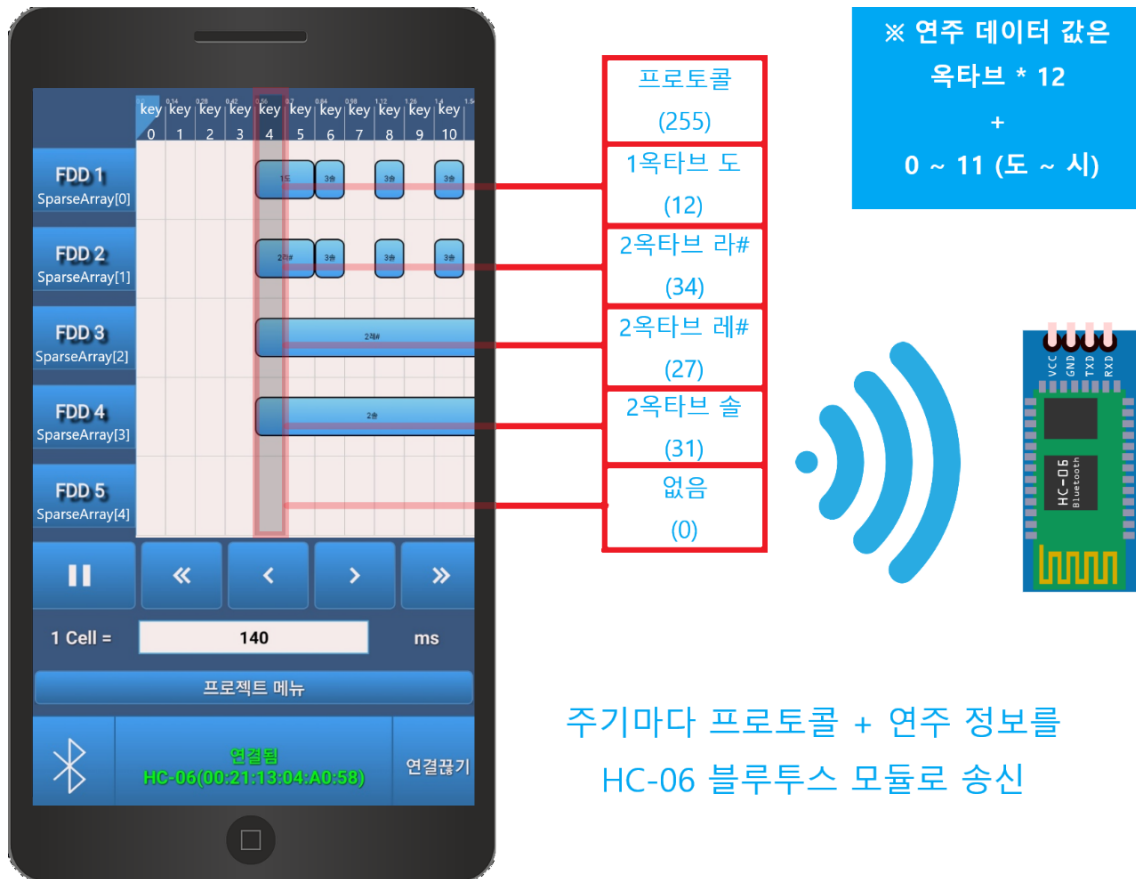


<그림 2.5>

연주 데이터를 작성한 후, 재생 버튼을 누르게 되면 주기 (140ms)마다 key 값에 증가시키고 각 SparseArray의 key 값에 대응되는 value 값 (Note 객체)를 받아와서 Note 객체에 담긴 음계 정보를 블루투스로 송신하게 됩니다.

2. 블루투스 통신

블루투스 송신에는 깃허브에 있는 akexorcist의 BluetoothSPP 라이브러리를 사용했습니다.



<그림 2.6>

스마트폰은 주기마다 **프로토콜 + FDD n개의 연주 정보** (n + 1 바이트)를 BluetoothSPP 라이브러리의 send메소드로 송신합니다. 블루투스 모듈 HC-06 단에서는 프로토콜(255)의 수신을 계속 기다리다가, 255를 받게 되면 아두이노 보드메모리의 buffer[n]에 연주 정보 n개를 저장시킵니다.

	도	도#	레	레#	미	파	파#	솔	솔#	라	라#	시
1	32.7	34.6	36.7	38.9	41.2	43.65	46.25	49	51.9	55	58.25	61.75
2	65.4	69.2	73.4	77.8	82.4	87.3	92.5	98	103.8	110	116.5	123.5
3	130.8	138.4	146.8	155.6	164.8	174.6	185	196	207.6	220	233	247
4	261.6	146.8	293.6	311.2	329.6	349.2	370	392	415.2	440	466	494

아두이노는 buffer[n]에 저장된 연주정보를 위의 음계 주파수 표에 따라 주파수로 변환시켜주어야 하는데, 이것을 잘 보면 첫 항이 1옥타브 음계이며 공비가 2인 등비수열이라는 규칙을 찾을 수 있습니다.

예) 1도 = 32.7 , 2도 = 65.4(32.7*2) , 3도 = 130.8(32.7*2^2) ...

따라서 1옥타브의 주파수 값만 배열 frequency[12] (1도 ~ 1시의 주파수값) 에 저장해

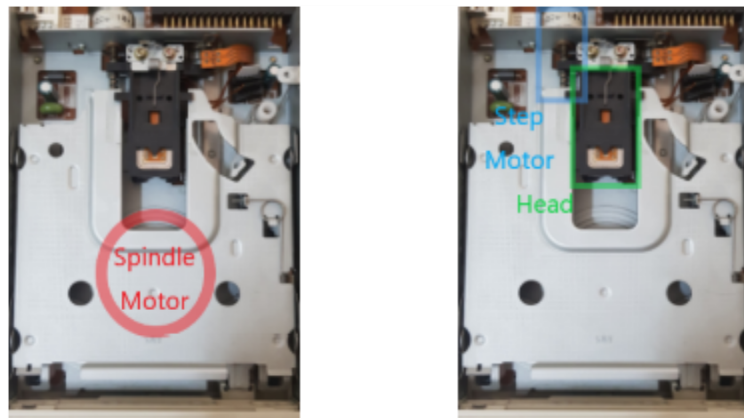
두면 나머지 옥타브의 주파수 값은 1옥타브 주파수 값을 이용한 공식으로 구할 수 있다는 소리가 됩니다.

연주 정보의 값은 $\text{옥타브} \times 12 + 0 \sim 11 (\text{음계})$ 의 형태로 되어있기 때문에 연주 정보로 다시 옥타브(연주 정보/12)와 음계 값(연주 정보%12)을 얻을 수 있으며 이것을 다음의 식으로 주파수로 변환할 수 있습니다.

$$\text{주파수} = \text{frequency}[\text{음계}] * 2^{(\text{옥타브} - 1)}$$

3. FDD로 소리 내기

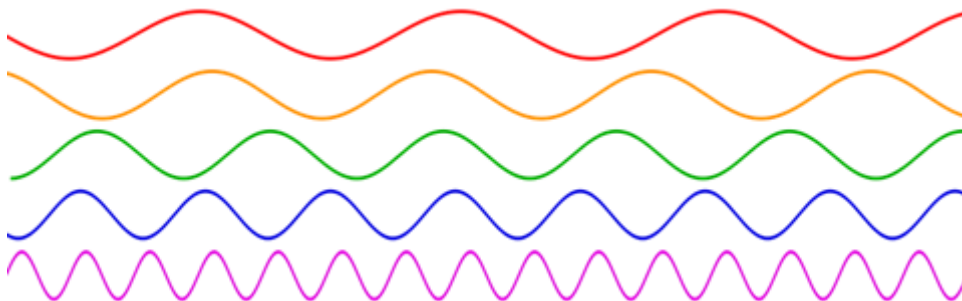
변환한 주파수 값으로 FDD를 동작시켜주기 전에, 이것이 어떤 원리로 소리를 내는지 알 필요가 있습니다. FDD의 기본적인 동작을 간단히 설명드리면,



<그림 2.7>

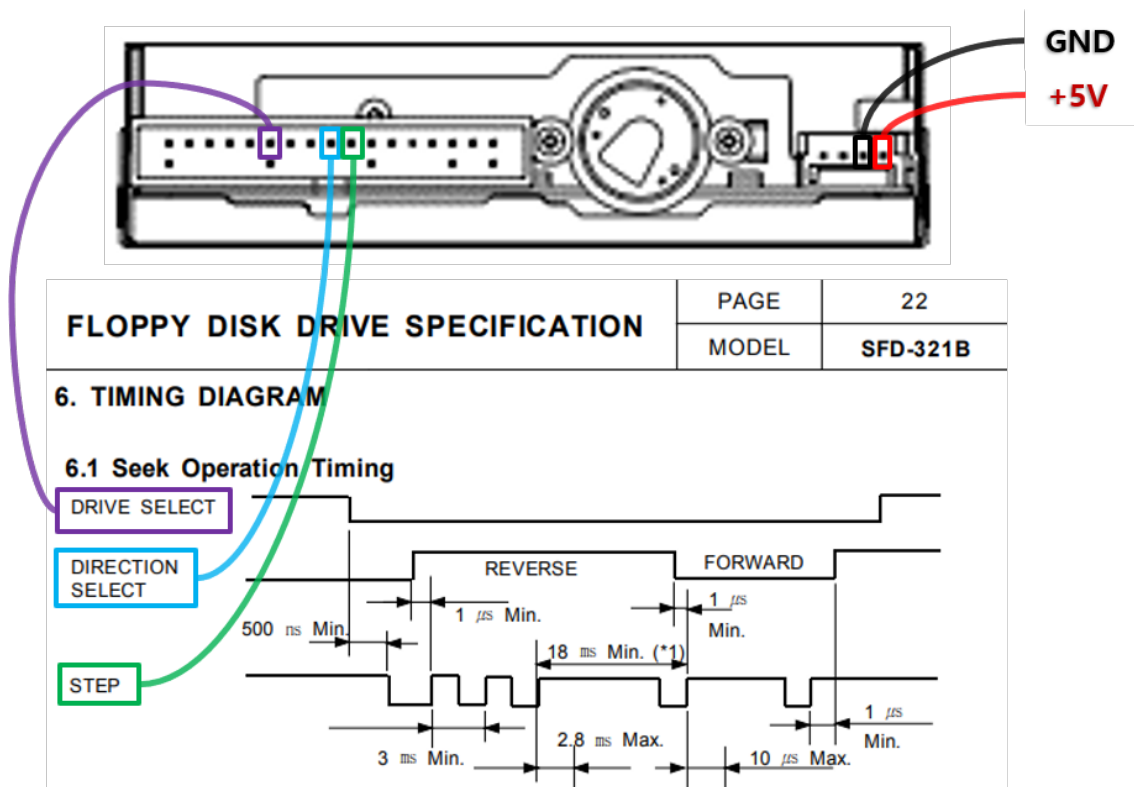
플로피 디스크를 넣으면, FDD의 **스핀들 모터**가 플로피 디스크 안의 원판을 회전시키고, 디스크가 회전하는 동안 FDD의 **스텝모터**가 **읽기/쓰기 헤드**를 이동시켜 디스크의 트랙을 바꿔 가면서 데이터를 읽거나 쓰는 것입니다.

그런데, 여기에서 **스텝모터**가 디스크의 트랙을 바꾸기 위해 **헤드**를 한번 움직일 때마다, “다각” 소리가 나게 됩니다.



<그림 2.8>

이것을 이용해서 1초 동안 헤드의 움직이는 빈도(주파수)를 조절해준다면, 주파수와 음계의 관계에 따라 서로 다른 높이의 음들을 출력하는 것이 가능해진다는 것이 되겠습니다. 그렇다면 헤드를 어떻게 움직일 지에 대해서 알아볼 필요가 있습니다.



<그림 2.9>

사용할 모델인 삼성 SFD-321B의 설명서에 이 동작이 타이밍 다이어그램으로 설명되어 있으며 일단 동작을 시키기 위해서는 +5V, GND가 전원과 연결되어 있어야 하고, 헤드를 움직일 때는 3개의 시그널 핀을 사용해야 합니다.

DRIVE SELECT핀은 드라이브의 인터페이스 신호를 활성화시키는 역할을 하기 때문에 가장 먼저 이 핀에 LOW를 주어야 다른 시그널 핀들을 쓸 수 있게 됩니다.

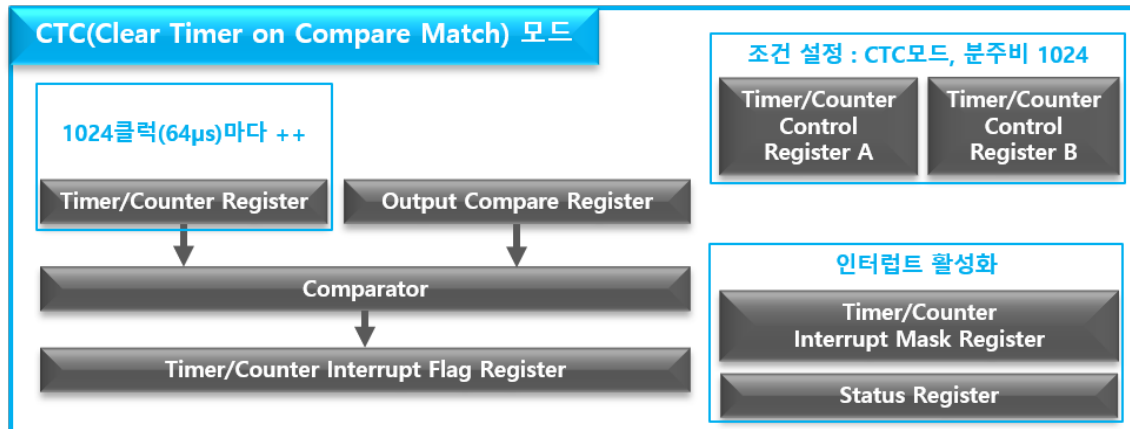
DIRECTION SELECT핀은 헤드를 움직일 때, 헤드가 움직이는 방향을 결정하는 역할을 하며, LOW일 시에는 디스크의 안쪽, HIGH일 시에는 바깥쪽으로 움직이게 됩니다.

STEP핀은 헤드를 움직이는 역할을 하며, LOW에서 HIGH로 변하는 순간(상승 에지)에 한번 헤드를 움직이게 합니다.

이것으로 헤드를 왕복시키면서 소리를 내게 만들려면, 프로그램이 시작(setup)될 때 **DIRECTION SELECT**핀에 HIGH나 LOW를 주고 **STEP**핀에 상승 에지를 80번 (삼성 SFD-321B는 3.5인치 FDD여서 디스크의 트랙이 80개이기 때문에) 발생시켜 헤드 위치를 초기화합니다. 이 다음부터는 1초마다 **STEP**핀을 연주할 음계의 주파수만큼 진동시켜 헤드를 움직이게 하고, 헤드가 한 번 움직일 때마다 카운트(헤드 위치 저장 변수)를 세서 카운트가 80이 될 때마다(헤드가 트랙의 양쪽 끝에 도달할 때마다) 카운트를 0으로 초기화하고 **DIRECTION SELECT**핀을 토글시키게 하면 되겠습니다.

4. 타이머 인터럽트

STEP핀에 1초마다 원하는 횟수만큼의 동작을 해 주려면, 아두이노의 마이크로 컨트롤러 내부에 있는 타이머/카운터 레지스터의 값을 적절하게 설정하면 됩니다. 타이머/카운터 레지스터는 CPU클럭마다 카운트를 세고, 카운트 값이 설정한 조건에 맞는 값이 되면 인터럽트가 발생하게 하기 때문에 이것으로 정밀한 시간 제어가 가능합니다. 아두이노의 delay, millis와 같은 시간 관련 함수들도 이것을 이용해서 구현합니다.



<그림 2.10>

원래는 CPU에서 1클럭($0.0000000625s = 62.5ns = 16MHz$ = 아두이노 메가의 클럭주파수)을 받을 때마다 Timer/Counter Register(TCNT)를 1씩 증가시키지만 TCNT가 1클럭에 1씩 증가하는 것은 너무 빠르기 때문에 TCCRA(Timer/Counter Control Register A), TCCRB(Timer/Counter Control Register B)의 분주비를 1024로 설정해서 TCNT가 1024클럭($0.000064s = 64\mu s$)에 1씩 증가하도록 속도를 늦춰주었으며, 동작 모드를 CTC(Clear Timer on Compare Match)모드로 설정해서 TCNT의 값이 점점 증가하여 Output Compare Register(OCRA)의 값과 같아질 때마다, TCNT를 Clear시키고 Timer/Counter Interrupt Mask Register (TIMSK)와 Status Register(SREG)의 인터럽트 비트가 허용되어 있다면 인터럽트가 발생되도록 하였습니다. 이것은 OCRA 레지스터의 값을 어떻게 설정하느냐에 따라 인터럽트의 발생 주기가 달라진다는 말이 됩니다.

1초에 발생해야할 인터럽트의 횟수는 **연주할 음계의 주파수*2** 입니다. 인터럽트가 주파수의 2배 발생되어야 하는 이유는 HIGH와 LOW를 번갈아가면서 토글시켜야 하기 때문입니다. 고로 인터럽트는 $1 / (\text{연주할 음계의 주파수} * 2)$ 초에 한 번 발생되어야 하며 TCNT값이 0부터 시작해 0.000064초마다 1씩 증가해서 $1 / (\text{연주할 음계의 주파수} * 2)$ 초가 되었을 때 도달하게 되는 값을 OCRA 레지스터의 값으로 설정해주면 되겠습니다.

$$OCRA = 1 / (\text{연주할 음계의 주파수} * 2) / 0.000064$$

이제 OCRA에 계산한 값을 넣어주게 되면 인터럽트 서비스 루틴이 1초에 주파수의 2배만큼 호출되게 되며 루틴이 호출될 때마다 **STEP핀**을 토글시키고, **STEP핀**이 HIGH가 될 때마다 헤드의 위치를 저장하는 변수를 증가시키고 확인해서 80이라면

DIRECTION SELECT핀을 토글시켜 헤드의 방향을 바꾸게 합니다. 이렇게 해서, FDD는

헤드를 왕복운동시키면서 설정한 주파수의 음을 내게 될 것입니다.

타이머 레지스터의 설정 값은 레지스터가 8비트인지 16비트인지에 따라 달라지게 되며, 타이머0, 2는 8비트 레지스터, 타이머1, 3, 4, 5는 16비트 레지스터입니다. 여기서 타이머0의 경우에는 아두이노에서 delay()나 millis()같은 함수의 구현에 이미 쓰고 있는 레지스터이기 때문에 타이머 1,2,3,4,5 를 사용했습니다.

8bit 레지스터 (Timer2)

TCCR2A |= (1<<WGM21) CTC 모드로 설정

Bit	7	6	5	4	3	2	1	0
Name	COM2A1	COM2A0	COM2B1	COM2B0	-	-	WGM21	WGM20
Value	0	0	0	0			1	0

TCCR2B |= (1<<CS22) | (1<<CS21) | (1<<CS20) 분주비 1024로 설정

Bit	7	6	5	4	3	2	1	0
Name	FOC2A	FOC2B	-	-	WGM22	CS22	CS21	CS20
Value	0	0			0	1	1	1

TIMSK2 |= (1<<OCIE2A) 인터럽트 허용 TIMSK2 &= ~(1<<OCIE2A) 인터럽트 비허용(연주 중단할 때 사용)

Bit	7	6	5	4	3	2	1	0
Name	-	-	-	-	-	OCIE2B	OCIE2A	TOIE2
Value						0	1 or 0	0

TCNT2 1024클럭 당 1씩 자동 증가

Bit	7	6	5	4	3	2	1	0
Name	TCNT2 [7:0]							
Value	0 ~ 255							

OCR2A 원하는 음계에 따라 값 수정

Bit	7	6	5	4	3	2	1	0
Name	OCR2A [7:0]							
Value	0 ~ 255							

16bit 레지스터 (Timer1, Timer3, Timer4, Timer5)

TCCRnB |= (1<<CSn2) | (1<<CSn0) | (1<<WGMn2) CTC 모드로 설정, 분주비 1024로 설정

Bit	7	6	5	4	3	2	1	0
Name	ICNCn	ICESn	-	WGMn3	WGMn2	CSn2	CSn1	CSn0
Value	0	0		0	1	1	0	1

TIMSKn |= (1<<OCIEEnA) 인터럽트 허용 TIMSKn &= ~(1<<OCIEEnA) 인터럽트 비허용(연주 중단할 때 사용)

Bit	7	6	5	4	3	2	1	0
Name	-	-	ICIEEn	-	OCIEEnC	OCIEEnB	OCIEEnA	TOIEEn
Value			0		0	0	1 or 0	0

TCNTn 1024클럭 당 1씩 자동 증가

Bit	7	6	5	4	3	2	1	0
Name	TCNTn [15:8] (TCNTnH)							
	TCNTn [7:0] (TCNTnL)							
Value	0 ~ 65535							

OCRnA 원하는 음계에 따라 값 수정

Bit	7	6	5	4	3	2	1	0
Name	OCRnA [15:8] (OCRnAH)							
Value	OCRnA [7:0] (OCRnAL)							
	0 ~ 65535							

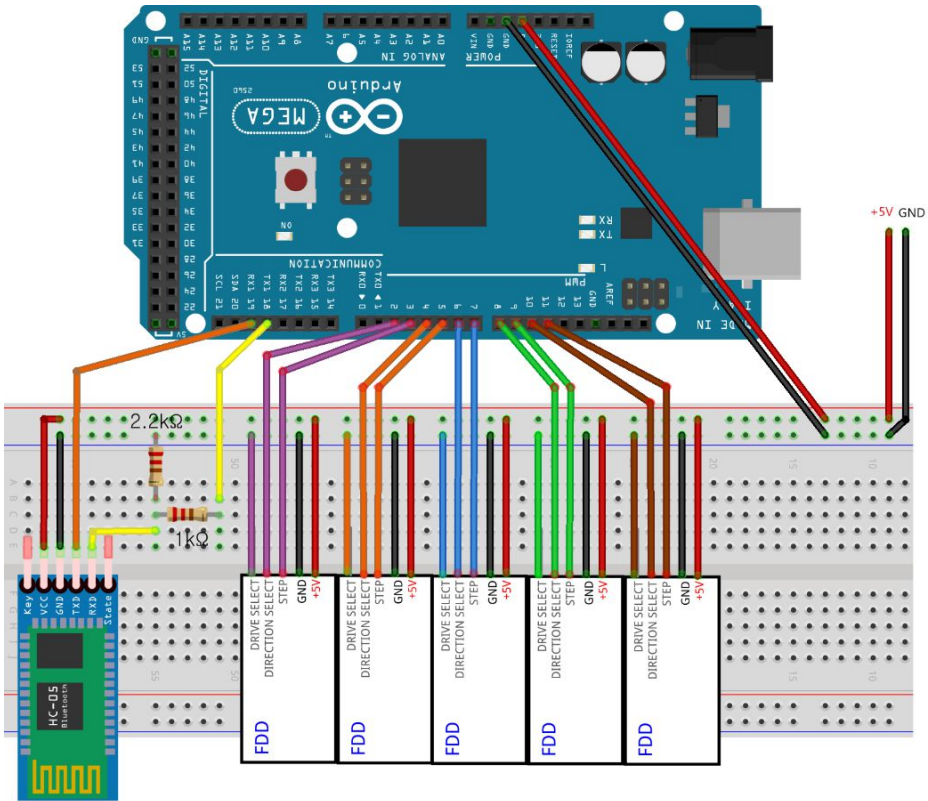
상태 레지스터 (Status Register)

SREG I= (1<<7) 상태 레지스터의 Global Interrupt Enable(I) Bit를 set시켜서 인터럽트 허용

Bit	7	6	5	4	3	2	1	0
Name	I	T	H	S	V	N	Z	C
Value	1	0	0	0	0	0	0	0

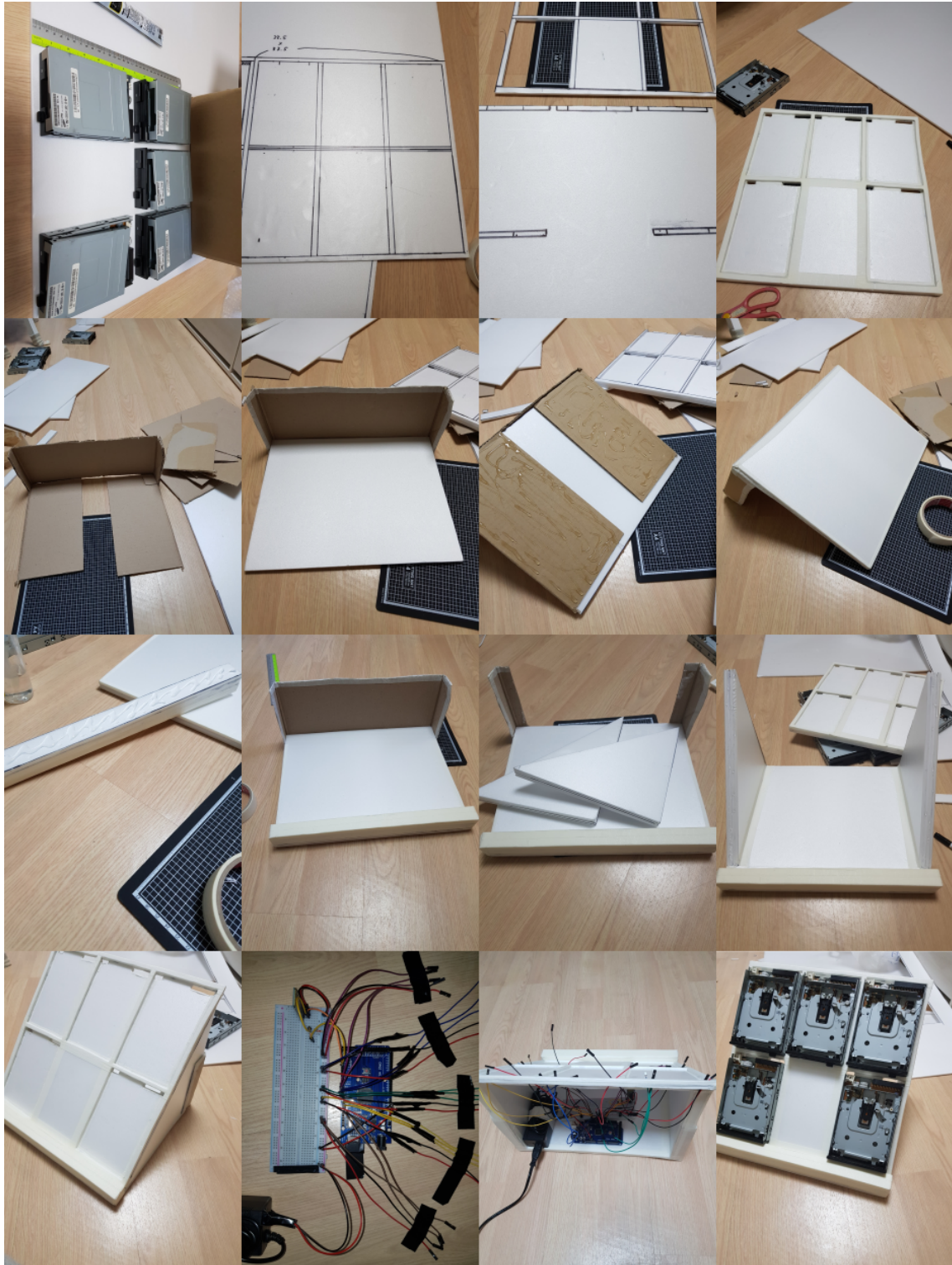
Ⅲ. 작품 구현 및 실험결과

1. 작품 설계



<그림 3.1>

2. 작품 제작

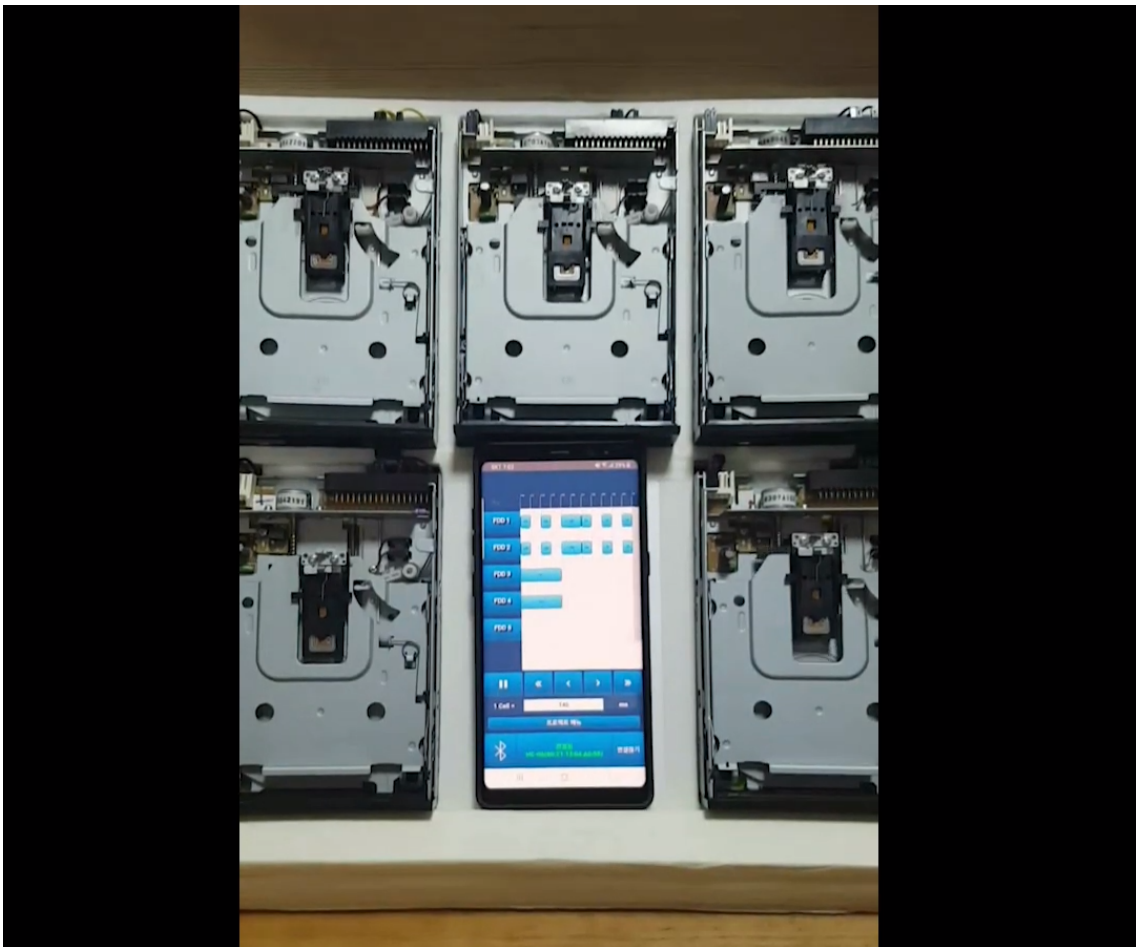


<그림 3.2>

IV. 결론

스마트폰을 이용한 블루투스 통신으로 다수의 FDD를 제어해 연주할 수 있게 되었습니다. 원래의 용도로는 쓰이지 않게 된 FDD가 음향 출력 장치로 재활용되는 것에 의미가 있으며, 헤드를 역동적으로 움직여가면서 소리를 내는 것은 피에조 부저와는 또 다르게 시각적인 재미까지 이끌어낼 수 있습니다.

동작 사진



<그림 4.1>

V. 참고 문헌

[1] 안드로이드 앱 프로그래밍 - 정재곤 지음, 이지스 퍼블리싱

[2] akexorcist의 BluetoothSPP 라이브러리

<https://github.com/akexorcist/Android-BluetoothSPPLibrary>

[3] 플로피 디스크의 순차적 읽기 동작

<https://m.blog.naver.com/PostView.nhn?blogId=tkdljs35&logNo=220774634175&proxyReferer=https%3A%2F%2Fwww.google.com%2F>

[4] 플로피 디스크 드라이브(SFD321B) 설명서

<http://www.techtravels.org/wp-content/uploads/pefiles/SAMSUNG-SFD321B-070103.pdf>

[5] Atmega 2560 (아두이노 메가) 데이터 시트

https://ww1.microchip.com/downloads/en/devicedoc/atmel-2549-8-bit-avr-microcontroller-atmega640-1280-1281-2560-2561_datasheet.pdf

[6] 마이크로컨트롤러 ATmega128 DIY 여행 - 신상석 지음, 창조와 지식

[7] 회로도 작성

<http://fritzing.org/home/>

[8] 외부 전원 연결

<https://www.technobyte.org/2016/07/power-up-the-arduino-uno/>

[9] 블루투스 HC-06 모듈 연결

<https://pinkwink.kr/1093>